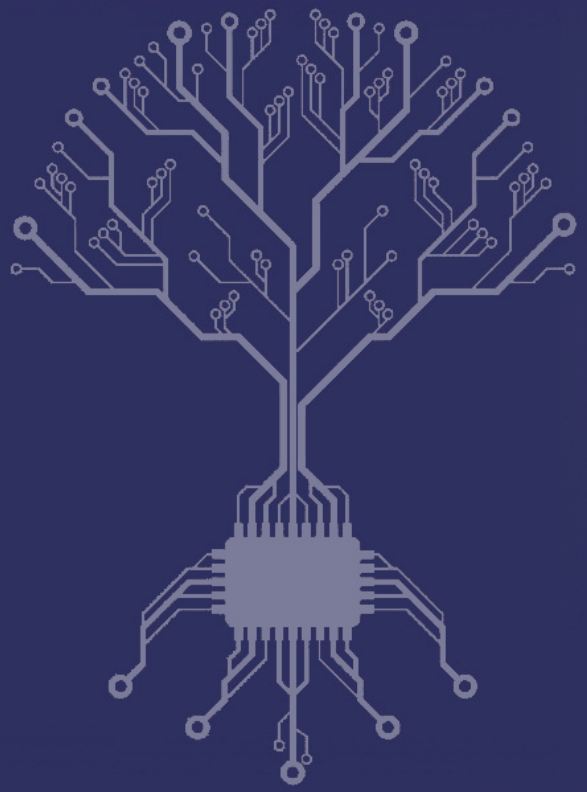
المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵜⵣⵔⴼⵜ ⴰⵏⴰⵎⵓⵔ
ⴷ ⵜⵓⵓⵔⵎⴰⵏ ⴰⵎⴰⵡⴰⵙ ⴷ ⵜⵙⵏⵙⵜ

Base de Données (1/3)

1 Introduction

2 SQL
- Langage de Définition de Données (*LDD*)
- Langage de Manipulation de Données (*LMD*)

Introduction



Après avoir établi le *MCD* et le *MLD*, il faut adapter ce dernier à un *SGBD* pour créer le *MPD* en utilisant un langage appelé *SQL* (Structured Query Language).

Système de Gestion de Base de Données

Un *SGBD* est un logiciel (*e.g. MySQL*, *Oracle*, *SQLite*, ...) qui permet de gérer une *BD* pour définir et manipuler les données, tout en garantissant leur cohérence et intégrité.

Le *SGBD* utilisé dans la suite est *SQLite*.

Selon le modèle relationnel, la base de données est composée d'un ensemble de relations (tables 2D) dans lesquelles sont placées les données, ainsi que les liens entre ces différentes relations.

Exemples :

student_id	first_name	last_name	field	average	city	center_code
1	Ali	Hassan	Mathematics	17.5	Rabat	CP101
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301
4	Laila	Ahmad	Economics	14.4	Rabat	CP102
5	Khalid	Ali	Mathematics	19.2	Salé	CP201

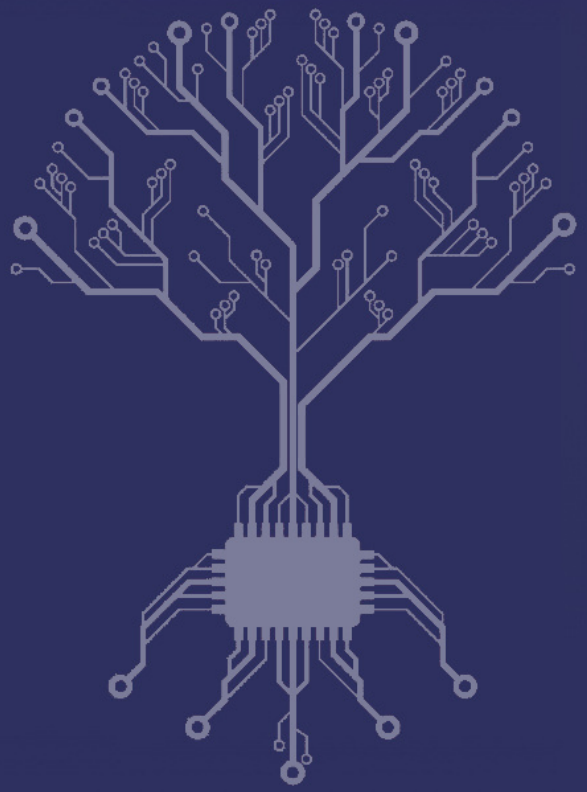
Table – Student

center_code	name	city
CP101	Moulay Youssef	Rabat
CP102	Omar Al Khayyam	Rabat
CP201	Selmane Al Farissi	Salé
CP301	Mohammed VI	Kénitra

Table – Center

Les tables ne représentent qu'une abstraction de l'enregistrement physique des données en mémoire.

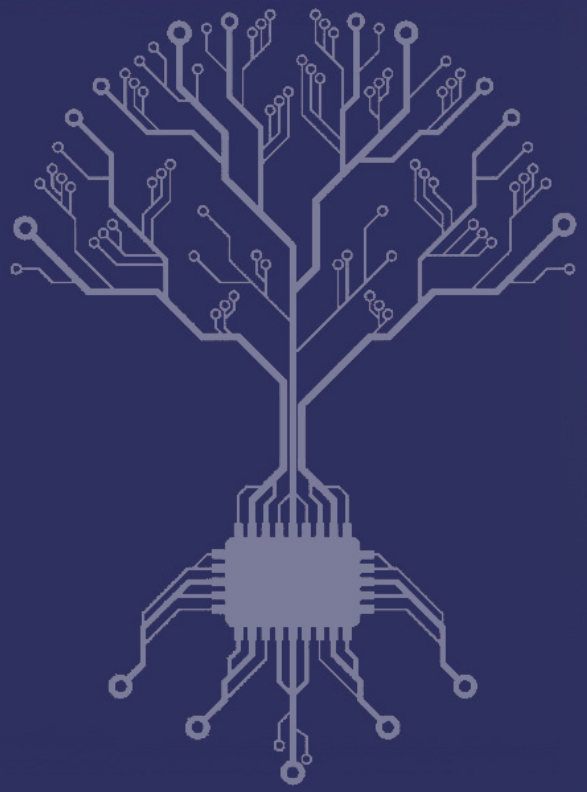
SQL



SQL est un langage informatique normalisé servant à exploiter des bases de données relationnelles. Il comporte plusieurs sous langages comme :

- Langage de Définition de Données (*LDD*)
- Langage de Manipulation de Données (*LMD*)
  - Langage d'Interrogation de Données (*LID*)

SQL :
Langage de Définition de Données



LDD

Le *LDD* permet de créer, de modifier et de supprimer une table.

Création

La création d'une table consiste à définir :

- Son nom ;
- Ses colonnes (leurs noms, types et contraintes d'intégrité) ;
- Ses contraintes d'intégrité.

Création

Syntaxe

```
CREATE TABLE [IF NOT EXISTS] nomTable (

nomColonne1 type contraintesColonne1,

nomColonne2 type contraintesColonne2,

...

contraintesTable,

...

);
```

Création : Types de données

- TEXT : Chaîne de caractères
- INTEGER : Nombre entier
- REAL : Nombre réel

Création : Contraintes d'intégrité

Définition

Une contrainte d'intégrité est une propriété que les données d'une base doivent vérifier pour garantir leur cohérence.

Création : Contraintes de colonne

- UNIQUE : Toutes les valeurs de la colonne doivent être différentes.
- NOT NULL ou NULL : Les valeurs de la colonne doivent être spécifiées ou non.
- PRIMARY KEY : Désigne la colonne comme clé primaire de la table.
- DEFAULT valeur : Permet de spécifier la valeur par défaut de la colonne.

Création : Contraintes de colonne

Exemple

La table *center*(*center_code*, *name*, *city*), où :

- *center_code* est une chaîne de caractères qui constitue la clé primaire,
- *name* et *city* sont des chaînes de caractères obligatoires,

peut être créée comme suit :

```
CREATE TABLE center (
center_code TEXT PRIMARY KEY,
name TEXT NOT NULL,
city TEXT NOT NULL
);
```

Création : Contraintes de colonne

- CHECK (condition) : Vérifie lors de l'insertion des enregistrements (lignes) que l'attribut vérifie la condition.

  Les opérateurs de comparaison :

  - $=, !=, <, >, <=, >=$
  - BETWEEN value1 AND value2
  - IN ou NOT IN (value1, value2, ...)
  - LIKE : permet d'utiliser des jokers (% pour une chaîne de caractères de longueur quelconque et _ pour un seul caractère)

Exemple

```
CREATE TABLE student (
student_id INTEGER PRIMARY KEY,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
field TEXT NOT NULL,
average REAL NOT NULL CHECK (average BETWEEN 0 AND 20),
city TEXT NOT NULL,
center_code TEXT NOT NULL
);
```

Création : Contraintes de table

Les contraintes de tables portent sur plusieurs attributs de la table sur laquelle elles sont définies :

- PRIMARY KEY (colonne1, colonne2, ...) : Désigne la concaténation des attributs cités comme clé primaire de la table.
- FOREIGN KEY (colonne) REFERENCES table (colonne) : Identifie une ou plusieurs colonnes comme étant clé étrangère.

Création : Contraintes de table

Exemple

Soit les schémas :

- professor(professor_id, first_name, last_name, mail)
- course(course_id, name, field)
- session(#professor_id, #course_id, date)

Création : Contraintes de table

Exemple

```
CREATE TABLE professor(
professor_id INTEGER PRIMARY KEY,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
mail TEXT NOT NULL CHECK (mail LIKE '%@%.%')
);
```

Création : Contraintes de table

Exemple

```
CREATE TABLE course (
course_id INTEGER PRIMARY KEY,
name TEXT NOT NULL,
field TEXT NOT NULL
);
```

Création : Contraintes de table

Exemple

```
CREATE TABLE session(
professor_id INTEGER,
course_id INTEGER,
date TEXT,
PRIMARY KEY (professor_id, course_id),
FOREIGN KEY (professor_id) REFERENCES professor (professor_id),
FOREIGN KEY (course_id) REFERENCES course (course_id)
);
```

Modification

- Ajout d'une colonne
  - Syntaxe : ALTER TABLE nomTable ADD (nomColonne type contraintes) ;
  - Exemple : ALTER TABLE student ADD (age INTEGER) ;
- Renommage d'une colonne
  - Syntaxe : ALTER TABLE nomTable RENAME ancienNom TO nouveauNom ;
- Suppression d'une colonne
  - Syntaxe : ALTER TABLE nomTable DROP nomColonne ;
  - Exemple : ALTER TABLE student DROP age ;

Suppression

```
DROP TABLE nomTable;
```

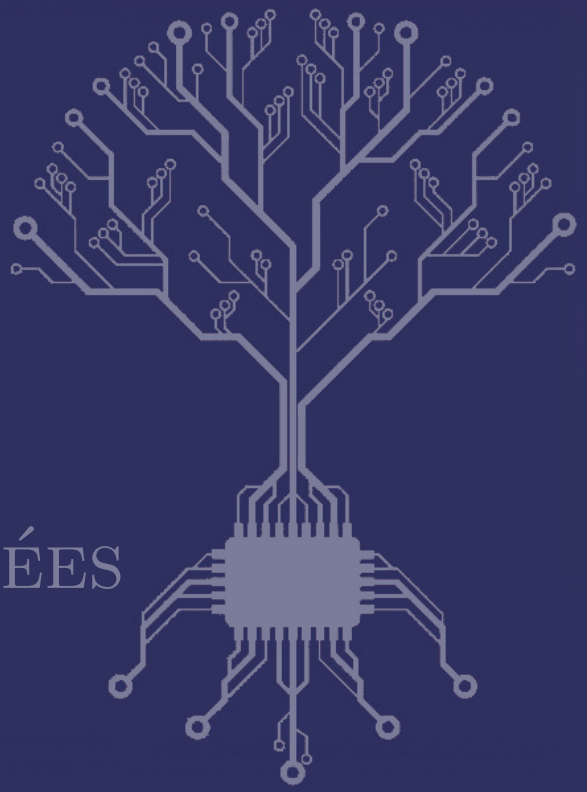
Modification et Suppression

La modification ou suppression d'une ligne dans une table peut être impossible s'il existe des lignes de clés étrangères dans d'autres tables référençant la clé primaire de cette ligne. Pour éviter ce genre de problème, on peut ajouter à la contrainte FOREIGN KEY des options :

Modification et Suppression

- En cas de modification :
  - ON UPDATE CASCADE : modifier aussi les valeurs des clés étrangères.
  - ON UPDATE SET NULL : mettre NULL à la place des valeurs des clés étrangères.
- En cas de suppression :
  - ON DELETE CASCADE : supprimer les lignes concernées.
  - ON DELETE SET NULL : mettre NULL à la place des valeurs des clés étrangères.

SQL :
Langage de Manipulation de Données



Le *LMD* permet d'insérer, de modifier et de supprimer les lignes d'une table.

Insertion de lignes

La commande INSERT permet d'insérer une ligne dans une table en spécifiant les valeurs à insérer.

Syntaxe

```
INSERT INTO nomTable VALUES
(valeur11, valeur12, . . . ),
(valeur21, valeur22, . . . ),
... ;
```

Insertion de lignes

Exemple

INSERT INTO center VALUES
('CP101', 'Moulay Youssef', 'Rabat'),
('CP102', 'Omar Al Khayyam', 'Rabat');

INSERT INTO student VALUES
(1, 'Ali', 'Hassan', 'Mathematics', 17.5, 'Rabat', 'CP101'),
(2, 'Sarah', 'Mohammad', 'Economics', 18.3, 'Rabat', 'CP102');

Modification de lignes

La commande UPDATE permet de modifier les valeurs d'une ou plusieurs colonnes, dans une ou plusieurs lignes, selon une condition.

Syntaxe

```
UPDATE nomTable SET nomColonne = expression, ... WHERE condition ;
```

En l'absence d'une clause WHERE, toutes les lignes sont mises à jour.

Modification de lignes

Exemple

Soit la table *employee*(*employee_id*, *name*, *salary*, *departement*) ; Pour augmenter de 10% le salaire des employés du département commercial on exécute la requête :

UPDATE employee SET salary = salary * 1.1 WHERE departement = '*commerce*';

Suppression de lignes

Syntaxe

DELETE FROM nomTable WHERE condition ;

Exemple

La requête :

DELETE FROM employee WHERE departement $= 'commerce'$;

supprime tous les employés du département commercial.